

Gen AI Training

Capstone Project

ProjectLens:

1. Executive Summary

ProjectLens is an AI-driven technical auditing platform I built to evaluate student software projects against actual enterprise engineering benchmarks. While developing the core logic, I realized standard LLMs aren't reliable enough on their own, so I integrated a Retrieval-Augmented Generation (RAG) pipeline to provide accurate suitability scoring, architectural gap analysis, and tailored mock interviews.

2. Problem Statement

When looking at academic projects (including my own and my peers'), I noticed a massive gap: they almost always lack production-grade standards like ACID compliance, distributed systems logic, or proper security. If you try to use standard LLM chatbots for help, they just give generic coding advice instead of strict architectural audits. Because of this, students struggle to figure out the "gap" between their current tech stack and what top-tier tech companies actually expect them to know in an interview. I designed this platform to fix that disconnect.

3. Proposed Solution & Objectives

To solve this, my plan for ProjectLens focused on three main objectives:

- **Objective 1:** Build a deterministic auditing engine that compares a user's tech stack to specific industry domain requirements (like Fintech, E-Commerce, or Health-tech).
- **Objective 2:** Implement a localized vector database (ChromaDB) to ground the AI in factual engineering standards. This is a crucial step to prevent the AI from hallucinating or giving generic advice.
- **Objective 3:** Generate an automated "Interview Simulator" that tests the user specifically on the vulnerabilities and gaps found in their exact project design.

4. Technology Stack

Here is the stack I decided on and my reasoning while planning the architecture:

- **Frontend:** React.js and Tailwind CSS. I chose this because I need a modular, component-based UI, and Tailwind allows for really rapid styling so I can spend more time focusing on the backend logic.
- **Backend:** Node.js and Express.js. This was necessary for handling asynchronous external LLM API calls without blocking the event loop.
- **Database / Vector Store:** ChromaDB. I needed an efficient way to store and retrieve high-dimensional text embeddings locally for the RAG pipeline.
- **Generative AI:** Google Gemini 1.5 Flash API. I went with Gemini Flash because it is fast and handles JSON-enforced structured generation perfectly, which I need for the frontend to render the dashboard results.

5. System Architecture & Data Flow

I mapped out the system's data flow to work step-by-step like this:

1. **Client Request:** The user submits a JSON payload from the React frontend, which contains their project description, selected domain, and tech stack.
2. **Middleware Intercept:** Before anything else happens, the Node.js server intercepts the request. I wrote a custom masking function here to scrub Personally Identifiable Information (PII) like names and emails for security.
3. **Vector Retrieval:** The sanitized description is converted into embeddings and used to query ChromaDB for the most relevant domain-specific engineering standards.
4. **LLM Orchestration:** The backend then combines the user input, the context retrieved from ChromaDB, and my strict "System Persona" instructions into a single structured prompt. This payload gets sent to Gemini.
5. **Client Response:** Gemini returns a deterministic JSON object. The backend routes this JSON back to the React UI for visualization.

6. Advanced GenAI Implementation

This is where the core logic sits. I made sure not to just wrap an API, but to build an actual intelligence pipeline:

- **The RAG Pipeline:** I took PDF and text documents containing real enterprise standards, chunked them, and ingested them into ChromaDB. This creates a localized knowledge base that the system queries before ever talking to the LLM.
- **System Persona & Prompt Guardrails:** I strictly instructed the AI to act as a "Principal Software Engineer". I also forced it to return valid JSON. This prevents it from generating conversational fluff and breaking the frontend rendering.

- **Security Layer:** I built the PII scrubber right into the middleware to ensure enterprise data privacy standards are actually being thought about and implemented.

7. Core Platform Features

For the actual user experience, I focused on these core outputs:

- **Dynamic Suitability Scoring:** A percentage score that maps the user's project to their selected industry domain.
- **Architectural Gap Analysis:** Specific, senior-level recommendations. Instead of generic advice, it suggests things like message queues, caching layers, or load balancers depending on the stack.
- **Automated Interview Simulator:** It generates 3 highly targeted technical questions based *only* on the user's provided tech stack and identified gaps.

8. Future Scope & Scalability

While building, I kept thinking about how this scales into a commercial product (Phase 2). My roadmap looks like this:

- **GitHub REST API Integration:** Moving away from text descriptions to actual static code analysis by autonomously parsing the user's `package.json` and folder structures directly from their repository.
- **Multi-Agent Workflow:** Implementing separate AI agents. For example, having a "Security Auditor" agent and a "Database Auditor" agent evaluate the code separately and then compile a joint report.
- **Cloud Deployment:** Containerizing the backend and vector store via Docker so it can be deployed at scale on cloud infrastructure.

9. Conclusion

Ultimately, ProjectLens successfully demonstrates the power of Applied AI (AIOps). By combining a traditional web architecture with a sophisticated RAG pipeline, I was able to solve a highly complex, domain-specific problem for engineering students trying to hit real-world enterprise standards.